Middle East Technical University  Department of Computer Engineering

# CENG 469

## Computer Graphics II

Spring 2025-2026
Assignment 2
Environment Mapping

Due date: April 29, 2026, Sunday, 23:59

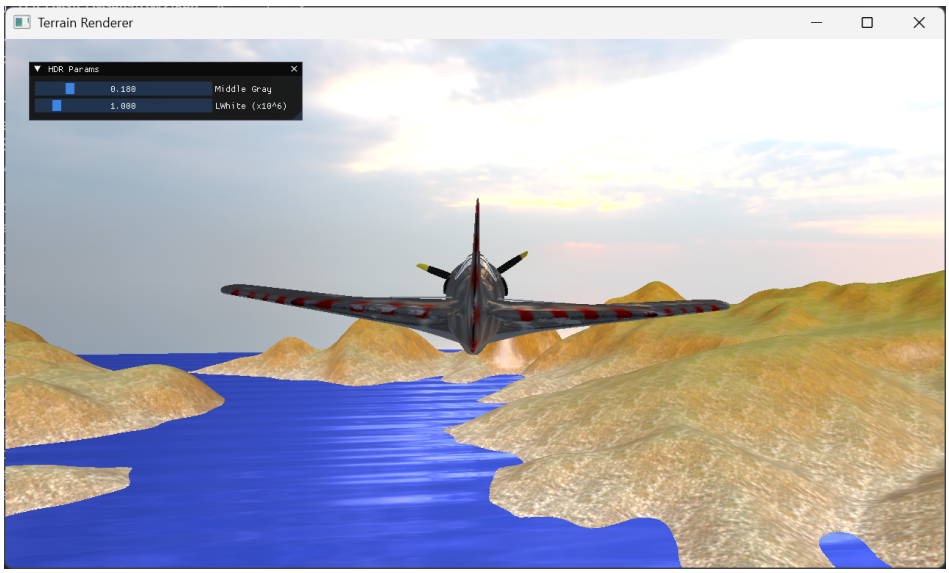


Figure 1: General View of the Assignment.

# 1 Objectives

In this assignment, you will continue implementing from the previous assignment and improve upon the generated terrain by adding texturing and environment mapping. You will shade the scene using an HDR texture (and additional directional lights if you wish) and apply a global tone mapping operator up on the final image. A plane will be rendered, and previous yaw, pitch, and roll rotation will be applied to the plane.

For the students who did not complete the previous assignment, the reference implementation of HW1 will be provided as a starter code.

The expected end product of this assignment is an OpenGL program which:

- Renders the previously terrain as described on the HW1. With additional texturing and shading.
- The user should be able to fly a plane through the terrain via flight controls as described on the HW1. These controls will be applied to the plane. An orbiting camera (3rd person camera) should be implemented that follows the plane. The submitted code will be checked for proper implementation.
- A reflective water plane that is animated with a basic cosine wave. This reflects the HDR image.
- A properly applied global tone mapping operator over the shaded image.

The specifications and the details are explained in Section 2. Note that you are free to implement your own design choices as long as you obey the requirements detailed in this assignment text.

# 2 Specifications

1. There will not be any arguments for the generated program.
2. It is not necessary to keep the previous combinations of the previous assignment. You can fine-tune the terrain parameters according to your scene.
3. The program should create a floating-point framebuffer to do the rendering in HDR space. This framebuffer should have four channels; the last channel (alpha channel) will hold the log luminance. Every shader should write its log luminance during shading.
4. All the rendering will be done over this framebuffer. After the frame rendering is done, this framebuffer will undergo tonemapping.
5. The HW1 terrain will be rendered with some improvements. These improvements are specifically unspecified and left for your artistic discretion. At least some form of texture mapping and using an HDR map is expected.
6. Reference implementation mixes multiple textures according to the elevation and slope of the surface. Each terrain texture also has its accompanying roughness texture to per-pixel vary the specular roughness. You may choose to do this approach or completely do a different approach with different atristic intents.

   > You can find many tiling textures and HDR images in https://polyhaven.com/.

7. An animated water plane will be rendered in addition to the terrain. The vertices of the water plane will be animated according to a cosine wave. **This should be done in the vertex shader.**

   > You can add multiple cosine waves to increase realism. With this approach, you will technically create a procedural noise.

8. Although not a requirement, you can expose the water animation parameters to the user via a GUI. To this end, the starter code (reference implementation of HW1) has Dear ImGUI dependency. If you continue from your own code, you may wish to merge that from the reference implementation.

9. Program will create a tessellated flat plane for the water. This generation should be done in the initialization time.

10. Water normals need to be calculated every frame since they are being animated. To calculate normals, there are multiple approaches:

    (a) **Do finite difference:** Assuming the Y is the up direction, get two other points towards the x and z directions. Calculate their own height values with your cosine animation. Subtract the resulting values from the animated vertex position. This will give the tangent and binormal vector. The cross product of these vectors will give the normal.

    (b) **Directly use the derivatives:** Since cosine is an analytical function, you can directly find its derivative, similar to HW1. Again, the cross product of these derivatives will give the normal.

11. As a background image, the HDR image itself should be rendered to the HDR framebuffer. You can directly map the HDR image to the view frustum by drawing a single quad that spans the entire view frustum. This is the classical approach to work over an image before the compute shaders. This approach:

    (a) Draws a single quad (or even a single triangle) that covers the entire NDC space (`[-1, 1]`).

    (b) sends NDC of each pixel to the fragment shader.

    (c) Fragment shader calculates the view direction of every pixel according to the camera parameters.

    (d) Spherical HDR map is queried according to the view vector.

12. A plane will be rendered and HW1's controls should be applied to the plane (Detailed input specifications will be explained later). The plane that is used on the reference implementation will be provided in the asset pack. The reference plane has 4 components: body, cockpit glass, propeller, and cables. These components are available as triangulated .obj files, and the MeshGL class should be able to load these. To properly assemble the plane, local to plane-local transformations are given below:

    (a) Body does not have any model transformations, plane directly looks towards +Z direction.

    (b) Propeller do require translation of `[0, 0, 13.719]` to be at its proper location. It sits at the origin.

    (c) Cockpit glass requires a translation of `[0, 2.733, -1.489]`.

    (d) Cables requires `[0, 3.644, -10.638]` translation.

13. Plane propeller needs to be rotated, you can achieve this effect via pre-rotating the propeller before doing any transformations.

14. You are free to change the plane with another plane as long as it has some form of animation on it (so that the implementation complexity stays the same).

15. Plane cockpit should reflect the HDR map, and all other components need to be textured (reference plane textures will be provided in the asset pack).

16. After frame is finalized, mipmap generation routine (`glGenerateMipmaps()`) will be called to find the average log luminance. The average value will be at the bottom-most mip (`1x1` mip). The technique is thoroughly described here.

    > If you already know and have used compute shaders, you can use compute shaders to find the average luminance afterwards. The approach described above assumes that compute shaders are not available or unknown by the implmenter.

17. As described in Item 11, a similar postprocess approach can be used to tonemap the framebuffer. In this case, you do not need to find the view directions but the UV coordinates for each pixel and query the framebuffer.

18. As for inputs, an orbiting camera will be implemented. This camera will be attached to the plane.

19. While holding the **Right Mouse Button**, horizontal and vertical mouse movements will orbit the camera around the plane. Please note that this is different from HW1, since the potential GUI usage will clash with the left mouse button.

20. **Mouse Wheel** will zoom in and out of the camera.

21. As for the plane inputs; **W** will increase the speed of the plane. **S** will decrease the speed. The plane only moves in its forward direction.

22. While holding the **Left Alt**, horizontal and vertical mouse movements will apply to the plane. Horizontal movement will do Yaw rotation. And vertical movement will do pitch rotation.

23. similar to the HW1, **E** and **Q** will do the yaw rotation. Unlike HW1, these will be applied to the plane instead of the camera.

24. **ENTER** button should toggle fullscreen mode on and off, similar to the HW1. Please check GLFW documentation for hints. Initially, the window should be in windowed mode.

25. There is no frames-per-second (FPS) constraint in the homework, but the user interaction should feel continuous and smooth except when generating the terrain mesh.

# 3 Blog Post

You should write a blog post that shows some output visuals from your work, and explains the difficulties you experienced, and design choices you made during the implementation phase. You can also write about:

- An analysis of **samples** vs **FPS** (frames-per-second) of your code (maybe as a table).
- Implementation of your tone mapping technique. which global operator did you use, its parametrization, and other related information.
- If you've decided to implement more advanced water animation, you can describe the technique and its parametrization.
- Some interesting design choices you made during implementation, etc.

The deadline for the blog post is 3-days later than the deadline **of your submission** (Unless you have submitted before the deadline, then you can submit 3-days later than the deadline). You consume your late days only if you submit your "code" late. However, submitting the blog post more than three days later will incur grade deductions.

# 4 Hints and Tips

1. Many of the parts of the HW is specifically left ambigious. Such as exact shading calculations using the HDR image, which shading model should you use, exact parametrization of the water animation, etc. These should be decided by you and should be explained on your blog post.
2. Try to start early. Although all of the baseline functionalities are simple and easy to implement, there are multiple functionalities. Try to implement the absolute baseline first, then try to improve your renderings.
3. TextureGL class on the HW1 starter code does not read HDR images properly. Please update with the new
4. While implementing the HW2, terrain generation on every debug session could be furstrating. You can pre-generate your terrain on a file and load it which should be faster. You can submit your code with this functionality as well.

# 5 Regulations

1. **Programming Language:** C/C++ for the main program and GLSL for shading computations is required. The starter code is given to you as such. Any C++ version can be used, and the starter CMake project sets up the C++ version as C++20.
2. **Changing the Code Template:** There are zero restrictions on modifying any code on the starter template. You can use extra libraries if you wish, given it does not defeat the purpose of the assignment (surface generation and rendering, for example). You can ask about the library that you want to use first via e-mail or in forums, and act accordingly.
3. **Groups:** All assignments are to be done individually.
4. **Submission:** Submissions will be done via ODTUClass. You should submit your blog post link to the Blog Links forum on ODTUClass, so that the URLs are publicly available for everyone's access, using the title "HW1 Blog Post". For code submission, create a "**tar.gz**" file named "hw1.tar.gz" that contains all your source code files, shader files, and a CMake

project. The executable should be named as "**main**" and should be able to be run using the following commands (any error in these steps may cause a grade deduction):

```
tar -xf hw1.tar.gz
cmake -B build . && make -C build
cd working_dir && ./TerrainRenderer
```

5. **Late Submission:** This course has a flexible late submission policy. For the entire semester, you have a total of 7 late days. For each homework, you can use at most 3 late days. Besides that, you can partition these days however you wish. Days can be used in an integer fashion (i.e., if you submit your work 4 hours late, an entire day's worth of credits will be used).

6. **Cheating: We have zero tolerance policy for cheating**. People involved in cheating will be punished according to the university regulations and will get 0 from the homework. You can discuss algorithmic choices, but sharing code between students is strictly forbidden. This applies to LLMs – you can use them for learning but not for code generation.
   Please be aware that there are advanced tools that detect similar and AI-generated codes even if the codes are edited.

7. **Forum:** Any updates/corrections and discussions regarding the homework will be on ODTUClass. You should check it on a daily basis. You can ask your homework-related questions on the forum of the homework on ODTUClass.

8. **Grading:** Your codes will be evaluated on Inek machines. We will not use automated grading, but evaluate your outputs visually. Implementing the homework as follows will get you 100 points:

   - The submitted project compiles properly.
   - Correctly implements the user-interaction capabilities.
   - It produces the expected rendering visuals with the correct surface calculations, normal calculations, and lighting calculations.
   - Proper global tonemapping operation.

   Different DTED files will be tested during grading.

   Note that you should test your code on an Inek machine before submission, as the frame rate might not be as high as your home computer if you have a powerful PC, and might cause hangs when launched. This might require reducing/optimizing per frame calculations in the code.

   Blog posts will be graded by focusing on the quality of the content, and a bonus can be given for extra efforts.

```
#include "thread_pool.h"

...

static constexpr size_t FLOAT_AMOUNT = 100;
// [10, 10, ..., 10]
std::vector<float> a(FLOAT_AMOUNT, 10);
// [15, 15, ..., 15]
std::vector<float> b(FLOAT_AMOUNT, 15);
// [X, X, ..., X] X = undefined
std::vector<float> c(FLOAT_AMOUNT);

// Create a thread pool with CPU's logical code count.
// Throughout the example lets assume thread count is 4
ThreadPool tp ({.threadCount = std::thread::hardware_concurrency()};

// Sumit FLOAT_AMOUNT of work to the thread pool.
// Thread pool will automatically partition these with
// respect to the 3rd argument and 2nd argument (lambda or
// a functor) will be called ONCE for each partitioned thread.
//
// In our example,
// thread_0 will get start = 0, end = 25.
// thread_1 will get start = 25, end = 50.
// thread_2 will get start = 50, end = 75.
// thread_3 (last thread) will get start = 75, end = 100.
tp.SubmitDetachedBlocks(FLOAT_AMOUNT,
                        [&](uint32_t start, uint32_t end)
    {
        for(uint32_t i = start; i < end; i++)
            c[i] = a[i] + b[i];
    }
    // There is a defaulted 3rd argument. If not provided,
    // work will be partitioned to all threads.
    // If provided, it will be partitioned into N threads where
    // N is the 3rd argument.
    // For debugging, you can set this to 1 so that your
    // breakpoints will not be hit by multiple threads.
    //, 1
    );
// Since operations are done in parallel, you can wait for the
// conclusion of the operations via 'Wait' function.
tp.Wait();

// After wait
// c = [25, 25, ..., 25]
```

Listing 1: Basic usage of ThreadPool class.